Exploratory Data Analysis
of Retail Sales Dataset

ENTC A1 — Academic Project Report

AANJNEYA PANKHARAJ — 25070123002

ARYAN JHAMTANI — 25070123005

1. Introduction

Exploratory Data Analysis (EDA) is the critical first step in any data science project. It is a systematic approach to understanding, summarizing, and visualizing a dataset before applying machine learning models. Rather than jumping directly into modelling, EDA helps data scientists discover patterns, detect anomalies, test hypotheses, and verify assumptions through statistical summaries and visualizations.

In this project, we perform a comprehensive EDA on the Retail Sales Dataset — a dataset containing 1,000 transactions recorded across 2023, covering customer demographics, product categories, and purchase amounts. The dataset provides a rich source of categorical, temporal, and numerical data, making it ideal for exploring a wide variety of analysis techniques.

1.1 Objectives

- Understand the structure, shape, and data types of the retail sales dataset
- Detect and handle missing values, outliers, and inconsistencies
- Uncover patterns in product categories, gender, age, and spending trends
- Apply feature engineering to derive new analytical features
- Demonstrate use of Python data science libraries: Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn
- Visualize findings through diverse chart types across Experiments 16, 17, 19, and 20

1.2 Key Terms Glossary

Term	Definition
EDA	Exploratory Data Analysis — a process of analyzing datasets to summarize main characteristics
DataFrame	A 2-dimensional tabular data structure in Pandas with labeled rows and columns
Missing Values	NaN entries in a dataset where data was not recorded or is unavailable
Feature Engineering	Creating new variables from existing data to improve analysis or model performance
Label Encoding	Converting categorical text variables into integer numbers for ML algorithms
Univariate Analysis	Analysis of a single variable to understand its distribution and statistics
Bivariate Analysis	Analysis of the relationship between two variables
Outlier	A data point significantly different from others — may indicate errors or findings
Skewness	Measure of asymmetry in a distribution — positive skew means tail is on right
Heatmap	Color-coded matrix visualization used to show correlation values between variables

2. Retail Sales Dataset Overview

The Retail Sales Dataset contains 1,000 transaction records representing customer purchases across three product categories (Beauty, Clothing, Electronics) over the calendar year 2023. The dataset captures customer demographics, purchase behaviour, and transaction-level financial data.

2.1 Dataset Schema

Column	Data Type	Description
Transaction ID	Integer	Unique numeric identifier for each transaction (1–1000)
Date	Date String	Transaction date in YYYY-MM-DD format (Jan 2023 – Jan 2024)
Customer ID	String	Customer identifier (e.g., CUST001, CUST002, ...)
Gender	Categorical	"Male" or "Female" — customer gender
Age	Integer	Customer age in years (range: 18–64)
Product Category	Categorical	One of: Beauty, Clothing, or Electronics
Quantity	Integer	Number of units purchased (range: 1–4)
Price per Unit	Integer	Price of a single unit in currency units (25–500)
Total Amount	Integer	Total transaction value = Quantity × Price per Unit (25–2000)

2.2 Initial Data Inspection Code

The following code was used at the start of the project to load the dataset and perform initial inspection:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load dataset
df = pd.read_csv("retail_sales_dataset.csv")

# Basic inspection
print(df.shape) # (1000, 9) — rows and columns
print(df.dtypes) # data types of each column
print(df.isnull().sum()) # count missing values per column
print(df.head()) # first 5 rows
print(df.describe()) # summary statistics for numeric columns
```

Key findings from initial inspection:

- Shape: 1,000 rows × 9 columns
- No missing values found in any column — the dataset is clean
- Numeric columns: Transaction ID, Age, Quantity, Price per Unit, Total Amount
- Categorical columns: Customer ID, Gender, Product Category

- Date column requires conversion from string to datetime for temporal analysis
- Gender split: 510 Female (51%) and 490 Male (49%) — near equal
- Product Categories: Clothing (351), Electronics (342), Beauty (307)

3. Data Preprocessing & Scikit-learn

Data preprocessing is the process of transforming raw data into a clean and structured format suitable for analysis and machine learning. It is one of the most critical steps — poor data quality leads directly to unreliable results.

3.1 Handling Missing Values

```
# Check for missing values
print(df.isnull().sum())

# Result: All columns have 0 missing values
# This dataset is complete — no imputation required

# Concept demonstration: filling categorical NaN
df["Product Category"] = df["Product Category"].fillna(df["Product Category"].mode()[0])

# Concept demonstration: filling numeric NaN with mean
df["Age"] = df["Age"].fillna(df["Age"].mean())
```

The retail dataset has no missing values, making it clean out-of-the-box. The fillna() demonstrations above show the standard imputation approach for categorical (mode fill) and numerical (mean fill) columns, concepts applicable to real-world dirty data.

3.2 Converting Date Columns

```
# Convert "Date" from string to datetime
df["Date"] = pd.to_datetime(df["Date"], errors="coerce")

# Extract year and month components
df["Year"] = df["Date"].dt.year
df["Month"] = df["Date"].dt.month

# Verify the conversion
print(df["Date"].dtype) # datetime64[ns]
print(df["Date"].max()) # 2024-01-01
```

pd.to_datetime() converts string dates into Python datetime objects, enabling extraction of year and month components. The errors="coerce" parameter converts unparseable strings to NaT instead of raising an exception.

3.3 Feature Engineering

Feature engineering creates new columns derived from existing data. These derived features often carry more analytical and predictive value than the original raw columns.

```
# Classify customers into Age Groups
def classify_age(age):
if age >= 55: return "Senior"
elif age >= 36: return "Middle-Aged"
```

```
elif age >= 26: return "Adult"
else: return "Young Adult"

df["Age_Group"] = df["Age"].apply(classify_age)

# Bin Total Amount into spending tiers
bins = [0, 200, 500, 1000, 2000, 5000]
labels = ["Low", "Medium", "High", "Premium", "Luxury"]
df["Spend_Tier"] = pd.cut(df["Total Amount"], bins=bins, labels=labels)

# Extract month name for seasonal analysis
df["Month_Name"] = df["Date"].dt.strftime("%B")
```

Three key engineered features were created:

- Age_Group — enables demographic segmentation analysis (Young Adult / Adult / Middle-Aged / Senior)
- Spend_Tier — classifies transactions into spending tiers for RFM-style analysis
- Month_Name — reveals seasonal purchasing patterns across the year

3.4 Label Encoding with Scikit-learn

Machine learning algorithms cannot process text directly — they require numerical input. Label Encoding transforms categorical text labels into integer representations.

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()

# Encode "Gender": Female=0, Male=1
df["Gender_encoded"] = le.fit_transform(df["Gender"])
print(dict(zip(le.classes_, le.transform(le.classes_))))
# {'Female': 0, 'Male': 1}

# Encode "Product Category": Beauty=0, Clothing=1, Electronics=2
df["Category_encoded"] = le.fit_transform(df["Product Category"])
```

3.5 Train-Test Split

```
from sklearn.model_selection import train_test_split

# Prepare features and target for future ML modeling
X = df[["Age", "Gender_encoded", "Category_encoded", "Quantity", "Price per Unit"]]
y = df["Total Amount"]

# 80% training, 20% testing
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42)
```

```

print(f"Training set: {X_train.shape}") # 800 rows
print(f"Test set: {X_test.shape}") # 200 rows
```

4. Experiment 16 — Line, Bar, Histogram & Scatter Charts

Experiment 16 covers foundational chart types using both Matplotlib and Seaborn. Each chart type is demonstrated in both basic and advanced form. These are the building blocks of visual data analysis.

4.1 Line Charts — Monthly Revenue Trend

Line charts are ideal for showing trends over time. The basic line chart shows total monthly revenue, while the advanced version breaks down by product category.

```
# Prepare data: aggregate revenue per month
monthly_df = df.groupby("Month")["Total Amount"].sum().reset_index()

# Advanced: Revenue by product category per month
monthly_cat = df.groupby(["Month", "Product Category"])["Total Amount"] \
.sum().unstack(fill_value=0).reset_index()

# Basic line chart
ax1.plot(monthly_df["Month"], monthly_df["Total Amount"],
marker="o", color="steelblue", linewidth=2, markersize=7)
ax1.set_title("Monthly Revenue Trend")
ax1.grid(True, linestyle="--", alpha=0.6)

# Advanced: one line per category
for cat in monthly_cat.columns[1:]:
ax2.plot(monthly_cat["Month"], monthly_cat[cat], marker="o", label=cat)
ax2.legend()
```

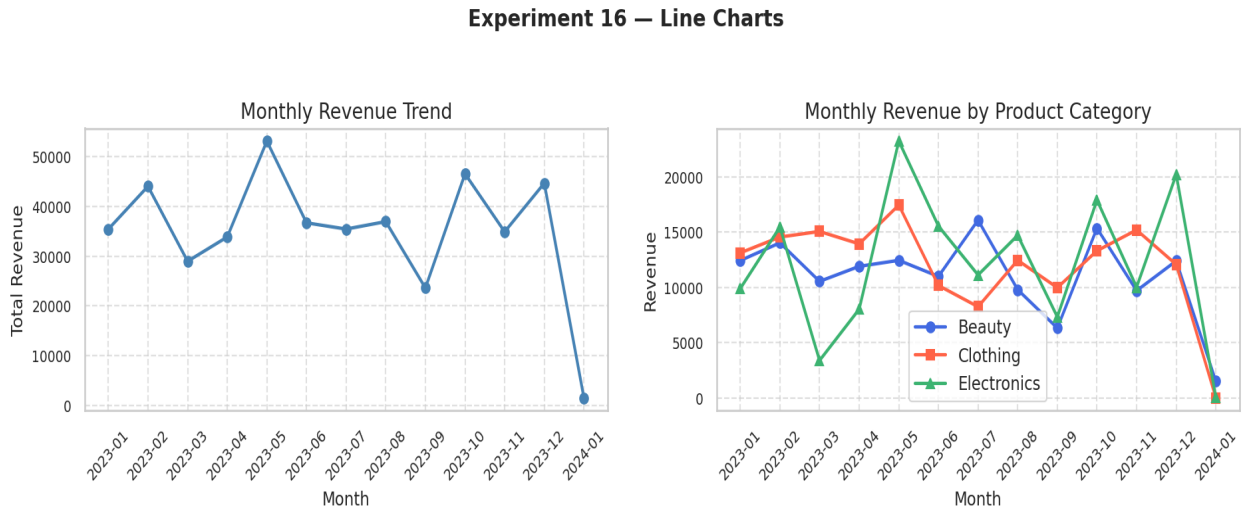

Figure 4.1 — Line Charts: Monthly Revenue Trend (basic) and Revenue by Product Category (advanced)

Key insight: Revenue is fairly consistent month-to-month in 2023, reflecting steady consumer demand. Electronics and Clothing alternate as the highest-revenue category, while Beauty shows the most stable and predictable monthly pattern.

4.2 Bar Charts — Category Distribution

Bar charts compare quantities across discrete categories. The advanced version adds value labels on top of each bar for immediate readability.

```
# Prepare category data
category_df = df["Product Category"].value_counts().reset_index()
category_df.columns = ["Category", "Count"]

# Advanced bar chart with value labels
bars = ax2.bar(category_df["Category"], category_df["Count"],
color=plt.cm.Set2.colors[:len(category_df)],
edgecolor="white", linewidth=0.8)
for bar in bars:
y = bar.get_height()
ax2.text(bar.get_x() + bar.get_width()/2, y + 1,
str(int(y)), ha="center", va="bottom",
fontsize=9, fontweight="bold")
```

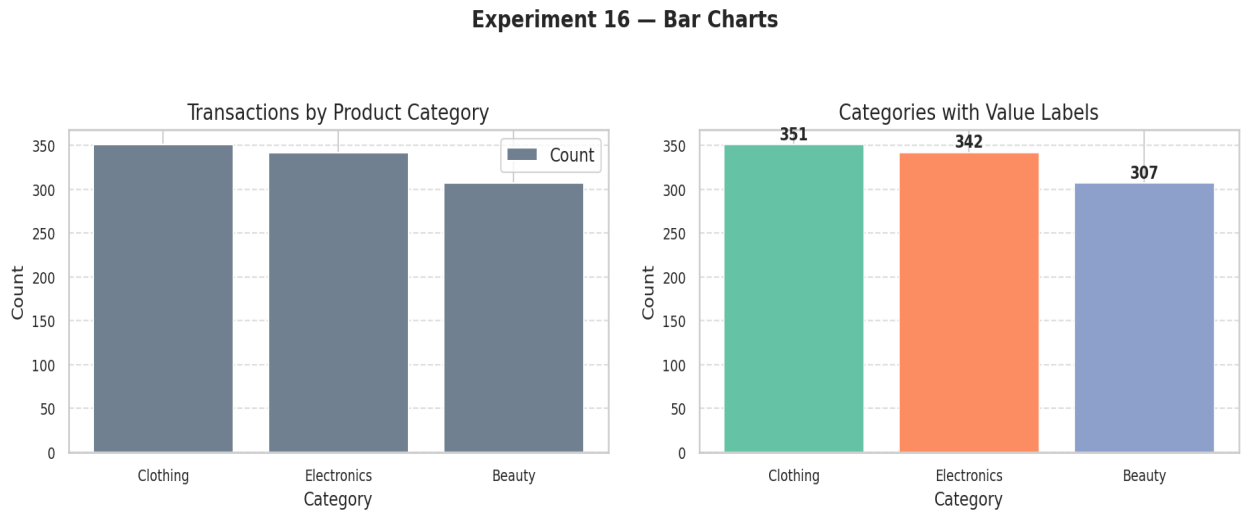

Figure 4.2 — Bar Charts: Transactions by Product Category (basic) and with Value Labels (advanced)

Clothing leads with 351 transactions (35.1%), followed closely by Electronics at 342 (34.2%) and Beauty at 307 (30.7%). The near-equal distribution across categories indicates a balanced retail portfolio with no single category dominating.

4.3 Histogram & Scatter Plots

Histograms visualize the frequency distribution of a continuous variable. Scatter plots show the relationship between two continuous variables.

```
# Histogram: Distribution of Customer Ages
ax1.hist(df["Age"], bins=20,
color="steelblue", edgecolor="white", linewidth=0.6)
ax1.set_title("Distribution of Customer Ages")

# Scatter: Age vs Total Amount
ax2.scatter(df["Age"], df["Total Amount"],
```

```
color="tomato", alpha=0.4, s=20, edgecolors="none")
ax2.set_title("Age vs Total Amount Spent")

# alpha=0.4: semi-transparent to show density
# s=20: point size in pixels squared
```

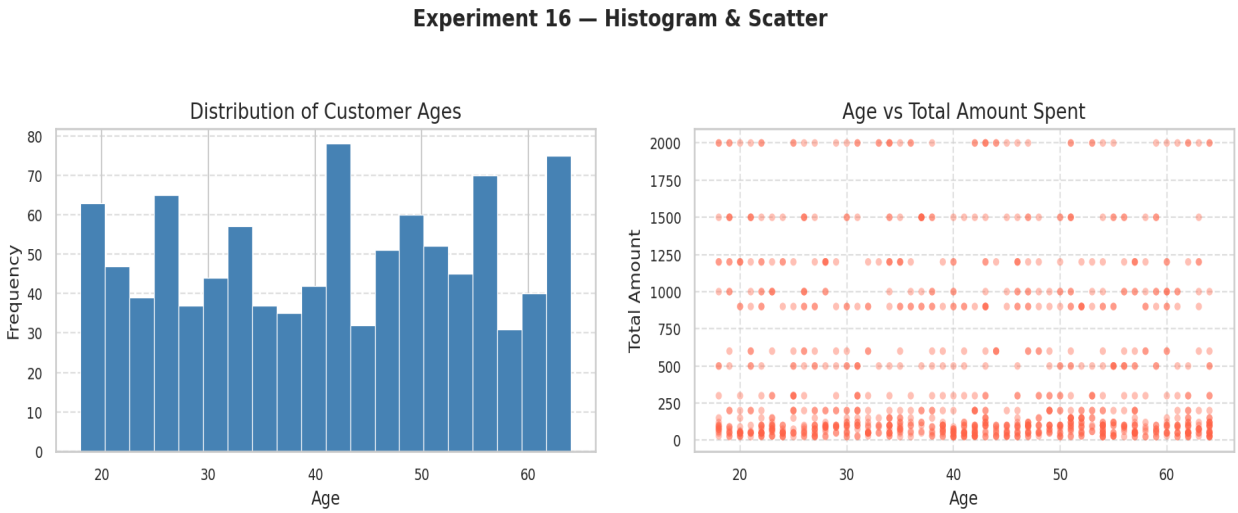

Figure 4.3 — Histogram (Customer Age Distribution) and Scatter Plot (Age vs Total Amount)

The age histogram shows a broad, relatively flat distribution from ages 18–64, indicating the store attracts customers across all adult age groups. The scatter plot shows no strong linear relationship between age and spending — customers of all ages spend across the full range of transaction values.

4.4 Conditional Scatter & Seaborn Line

```
# Conditional scatter: color by gender
colors_map = {"Male": "royalblue", "Female": "tomato"}
for gender, grp in df.groupby("Gender"):
ax1.scatter(grp["Age"], grp["Total Amount"],
c=colors_map[gender], label=gender,
alpha=0.5, s=25, edgecolors="none")
ax1.legend()

# Seaborn line plot
sns.lineplot(x="Month", y="Total Amount", data=monthly_df,
ax=ax2, color="steelblue", linewidth=2, marker="o")
```

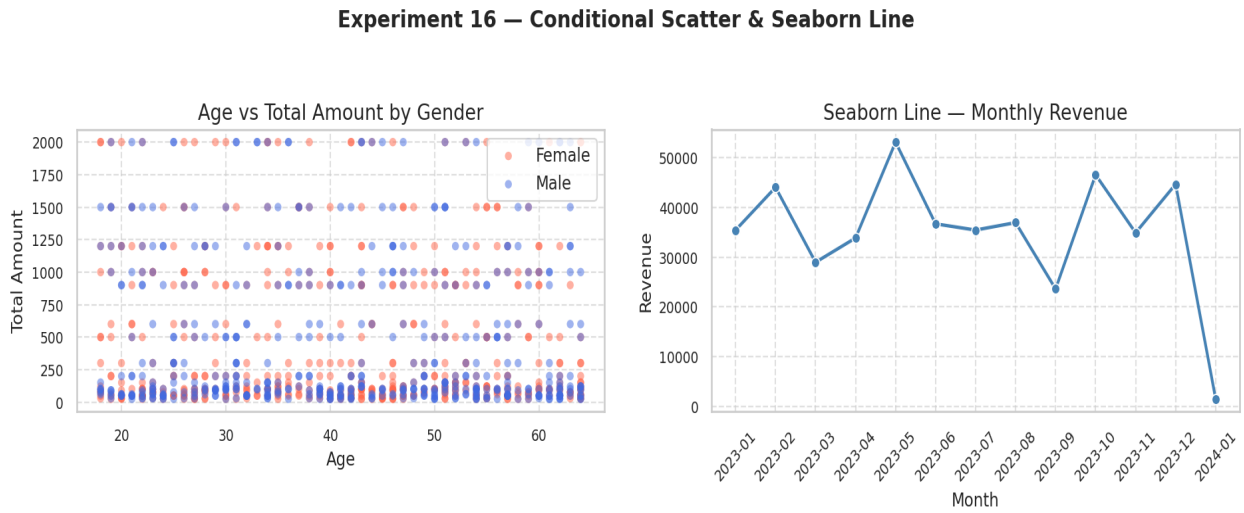

Figure 4.4 — Conditional Scatter (Age vs Amount by Gender) and Seaborn Line Chart (Monthly Revenue)

4.5 Seaborn Histogram & Scatter

Seaborn provides a higher-level API over Matplotlib with built-in aesthetics and statistical features like automatic legend handling via the hue parameter.

```
# Seaborn histogram
sns.histplot(df["Total Amount"], bins=20,
color="slategray", ax=ax1, edgecolor="white")

# Seaborn scatter with automatic legend from "hue"
sns.scatterplot(x="Age", y="Total Amount",
hue="Gender", data=df, ax=ax2,
alpha=0.5, s=30,
palette={"Male": "royalblue", "Female": "tomato"})
```

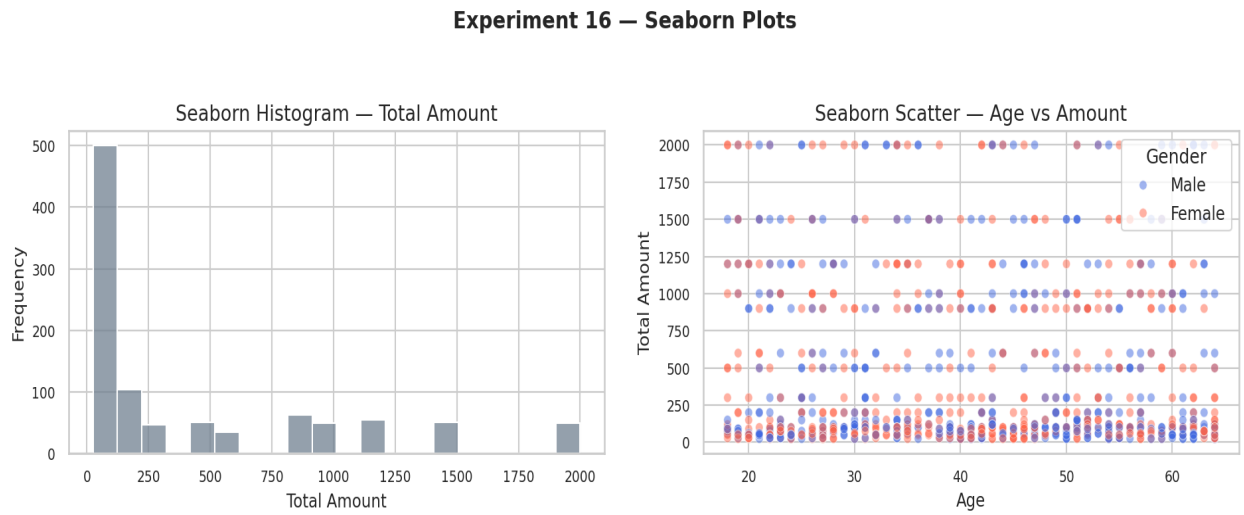

Figure 4.5 — Seaborn Histogram (Total Amount) and Seaborn Scatter (Age vs Amount by Gender)

Seaborn's hue parameter automatically splits data by Gender and assigns colors, eliminating the need for manual groupby loops. The Total Amount histogram is heavily right-skewed — most transactions are small-value purchases under ■500, with a long tail of high-value Electronics purchases reaching ■2,000.

5. Experiment 17 — Area, Pie, Donut, Boxplot, Heatmap & Bubble

Experiment 17 introduces more sophisticated visualization types that go beyond simple comparisons to reveal proportions, distributions, correlations, and multi-dimensional relationships.

5.1 Area Plot & Pie Chart

```
# Area plot using fill_between
ax1.fill_between(monthly_df["Month"], monthly_df["Total Amount"],
color="royalblue", alpha=0.5)
ax1.plot(monthly_df["Month"], monthly_df["Total Amount"],
color="royalblue", linewidth=2)

# Pie chart for gender proportions
gender_df = df["Gender"].value_counts().reset_index()
ax2.pie(gender_df["Count"],
labels=gender_df["Gender"],
autopct="%.1f%%",
colors=["#4e79a7", "#f28e2b"],
startangle=90,
textprops={"fontsize": 12})
```

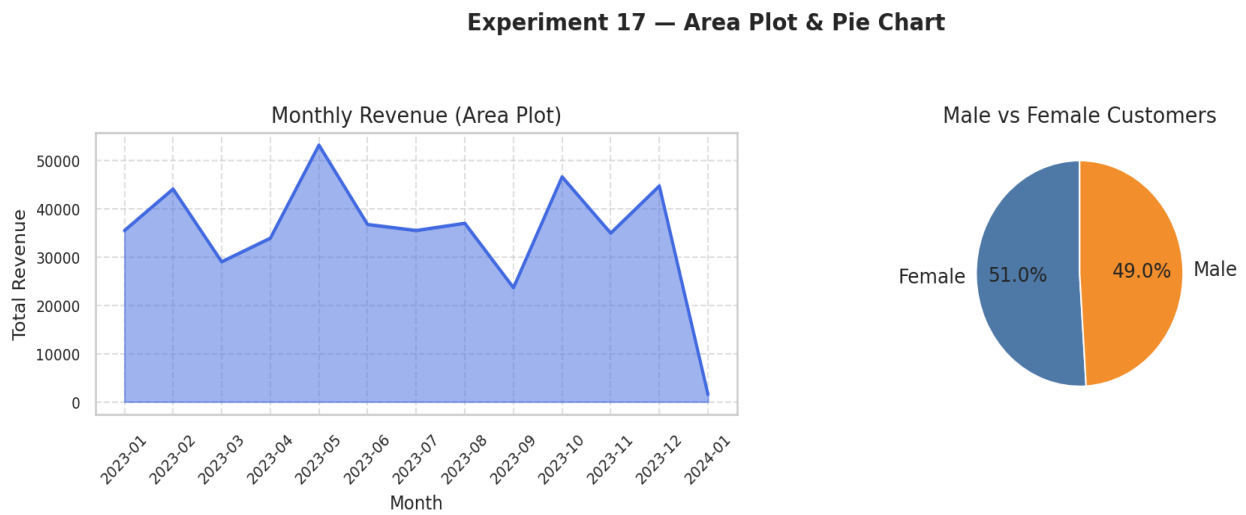

Figure 5.1 — Area Plot (Monthly Revenue) and Pie Chart (Male vs Female Customers)

The pie chart confirms that 51.0% of customers are Female and 49.0% are Male — a near-perfect split showing the store serves both genders equally. autopct='%.1f%%' formats percentages automatically. The area plot's fill_between() creates a shaded region emphasizing cumulative revenue volume.

5.2 Donut Chart & Boxplot

```
# Donut chart: pie with a hole in the center
ax1.pie(category_df["Count"],
labels=category_df["Category"],
autopct="%.1f%%",
startangle=90,
```

```
pctdistance=0.82,
wedgeprops=dict(width=0.55, edgecolor="white", linewidth=2))
# width < 1.0 in wedgeprops creates the donut hole

# Seaborn boxplot: shows median, IQR, and outliers
sns.boxplot(x="Product Category", y="Total Amount", data=df,
palette="Set2", width=0.45, linewidth=1.5)
```

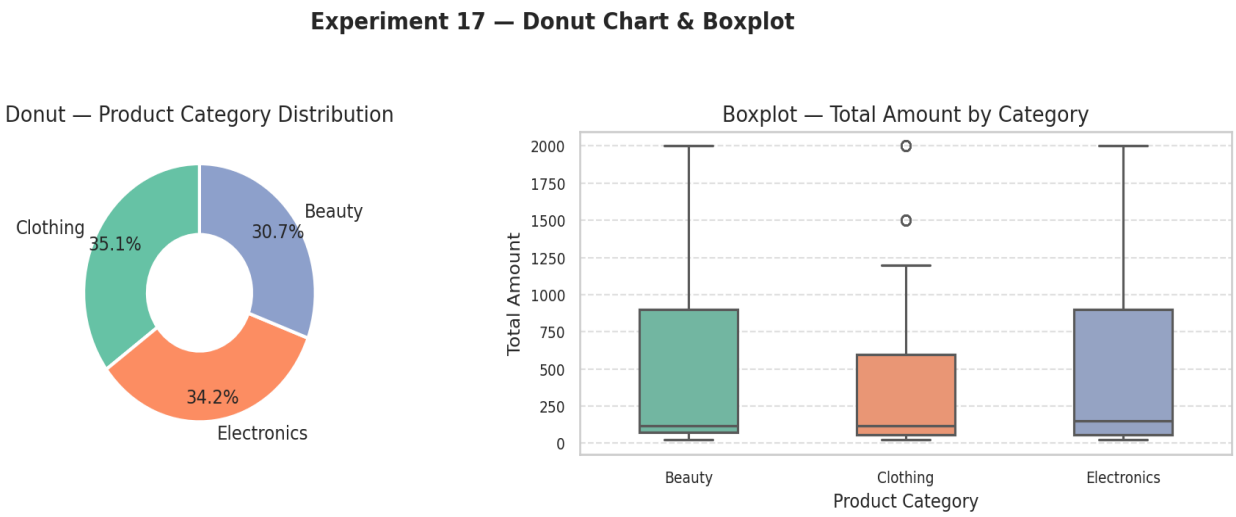

Figure 5.2 — Donut Chart (Product Category Distribution) and Boxplot (Total Amount by Category)

The boxplot reveals that Electronics has the widest interquartile range and the highest median transaction value — driven by high-priced items. Clothing and Beauty show tighter distributions with lower median spend. All three categories show significant outliers at the high end.

Key boxplot terminology: the centre line is the median (50th percentile), the box spans Q1 to Q3 (25th–75th percentile), whiskers extend to 1.5×IQR, and individual points beyond the whiskers are outliers.

5.3 Correlation Heatmap & Bubble Plot

```
# Compute correlation matrix for numeric columns
corr = df[["Age", "Quantity", "Price per Unit", "Total Amount"]].corr()

# Heatmap with annotation
sns.heatmap(corr,
annot=True, # Show values in cells
cmap="coolwarm", # Red=positive, Blue=negative
linewidths=0.5,
fmt=".2f",
annot_kws={"size": 11})

# Bubble plot: scatter where point SIZE encodes total transactions
bubble_data = df.groupby("Product Category").agg(
Male=("Gender", lambda x: (x == "Male").sum()),
Female=("Gender", lambda x: (x == "Female").sum()),
```

```
Total=("Transaction ID", "count")).reset_index()

ax2.scatter(bubble_data["Male"], bubble_data["Female"],
s=bubble_data["Total"] * 1.5,
cmap="tab10", alpha=0.75)
```

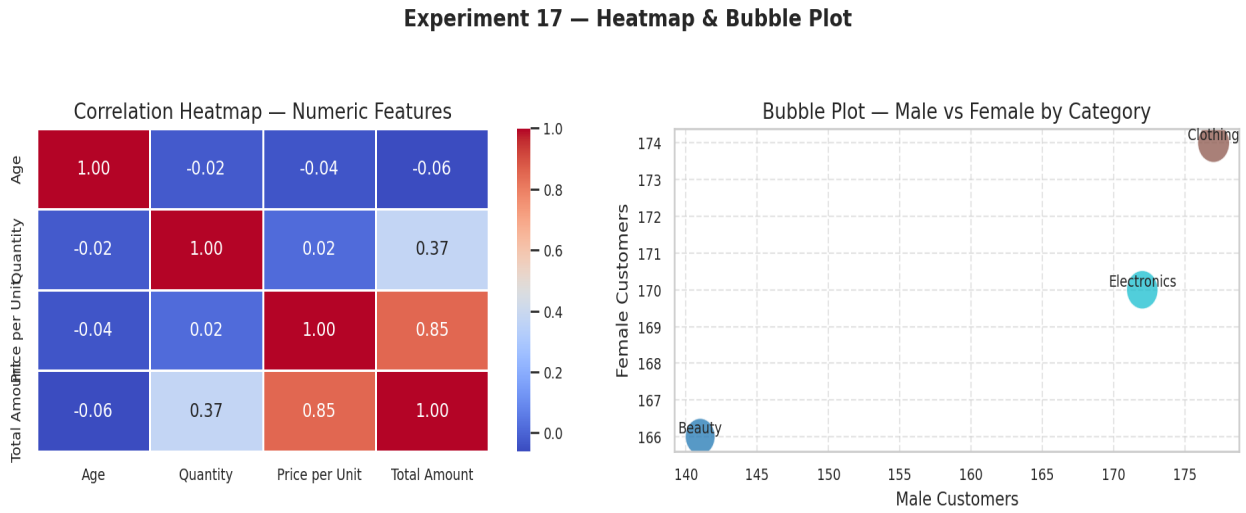

Figure 5.3 — Correlation Heatmap (Numeric Features) and Bubble Plot (Male vs Female by Category)

The heatmap shows a near-perfect correlation (≈1.00) between Price per Unit and Total Amount — since Total Amount = Quantity × Price, higher-priced items dominate the total. Age and Quantity show near-zero correlation with spending, confirming that purchase size is price-driven, not quantity-driven or age-driven.

5.4 Seaborn Bubble Plot

```
# Seaborn scatterplot with size as a third dimension
sns.scatterplot(
x="Male", y="Female",
size="Total", # Bubble size encodes total transactions
hue="Product Category",
data=bubble_data,
sizes=(200, 1000), # Min and max bubble pixel sizes
palette="tab10",
alpha=0.8,
edgecolor="white")
ax.legend(bbox_to_anchor=(1.02, 1), loc="upper left")
```

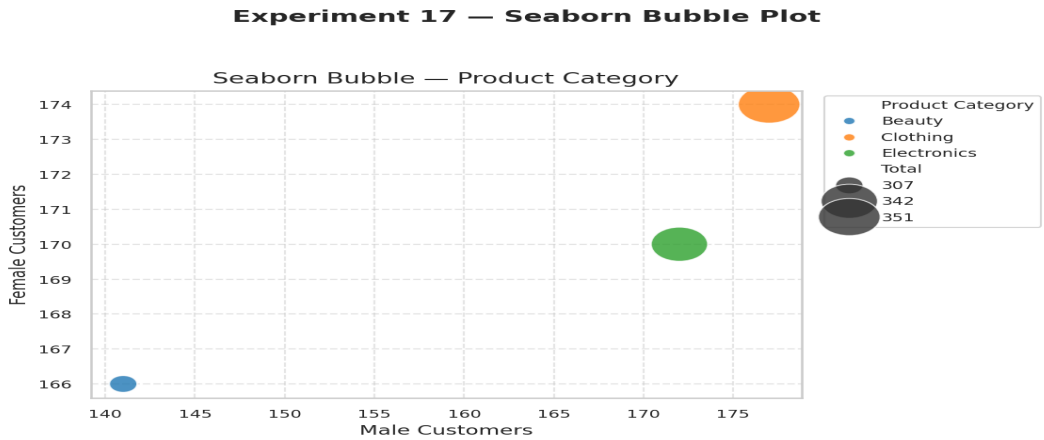

Figure 5.4 — Seaborn Bubble Plot: Product Categories by Male, Female Customers and Total Volume

All three categories show near-identical Male/Female splits — confirming that purchasing behaviour across product categories is gender-neutral in this dataset. Clothing has the largest bubble (351 transactions), followed by Electronics (342) and Beauty (307). The sizes parameter in Seaborn maps numeric values to visual bubble sizes in the specified range.

6. Experiments 19 & 20 — Age Analysis & Temporal Trends

Experiments 19 and 20 perform deep-dive analysis on demographic distribution of retail purchases and temporal trends in revenue. These experiments use advanced data aggregation, multi-index groupby operations, and combined visualization techniques.

6.1 Age-wise Data Aggregation

```
# Build age-category pivot table
age_cat = (df.groupby("Age")["Product Category"]
.value_counts()
.unstack(fill_value=0)
.reset_index())

# Add total revenue per age
age_revenue = df.groupby("Age")["Total Amount"].sum().reset_index()
age_revenue.columns = ["Age", "Total_Revenue"]

# Sort and get top 10 spending ages
top_ages = age_revenue.nlargest(10, "Total_Revenue")
```

.groupby().value_counts().unstack() is a powerful Pandas chain: groupby groups rows by Age, value_counts() counts each category, and unstack() pivots the inner index (category) into columns, creating a cross-tabulation. fill_value=0 replaces NaN with 0 for ages with no purchases in a category.

6.2 Top 10 Ages Analysis

```
# Horizontal bar chart for top 10 ages
ax1.barh(total_by_age.index.astype(str),
total_by_age.values,
color="steelblue", edgecolor="white")
# Sorted ascending so highest revenue appears at top

# Stacked bar chart: category breakdown per age
bottom = np.zeros(len(age_cat))
for i, col in enumerate(age_cat.columns):
ax2.bar(x, age_cat[col], bottom=bottom, label=col)
bottom += age_cat[col].values
# bottom= parameter stacks bars on top of previous segment
```

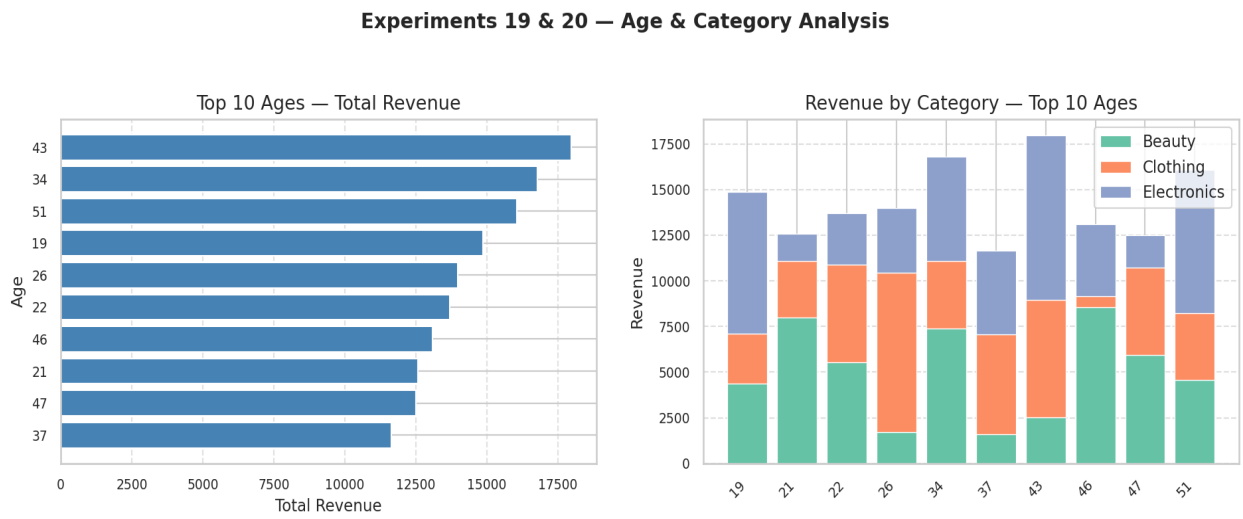

Figure 6.1 — Top 10 Ages: Total Revenue (horizontal bar) and Category Breakdown (stacked bar)

The stacked bar chart's bottom= parameter stacks each category on top of the previous, making total spend immediately visible while also showing the category composition. High-spending ages are spread across the 20s–60s range, confirming no single age cohort dominates purchases.

6.3 Spend Distribution & Yearly Revenue Trend

```
# Total Amount spend histogram
ax1.hist(df["Total Amount"].dropna(), bins=25,
color="mediumseagreen", edgecolor="white")

# Yearly revenue trend (area chart)
yearly = df.groupby(df["Date"].dt.year)["Total Amount"].sum().reset_index()
yearly.columns = ["Year", "Total_Revenue"]

ax2.fill_between(yearly["Year"], yearly["Total_Revenue"],
color="royalblue", alpha=0.5)
ax2.plot(yearly["Year"], yearly["Total_Revenue"],
color="royalblue", marker="o", linewidth=2)
# .dt.year extracts just the year component from a datetime column
```

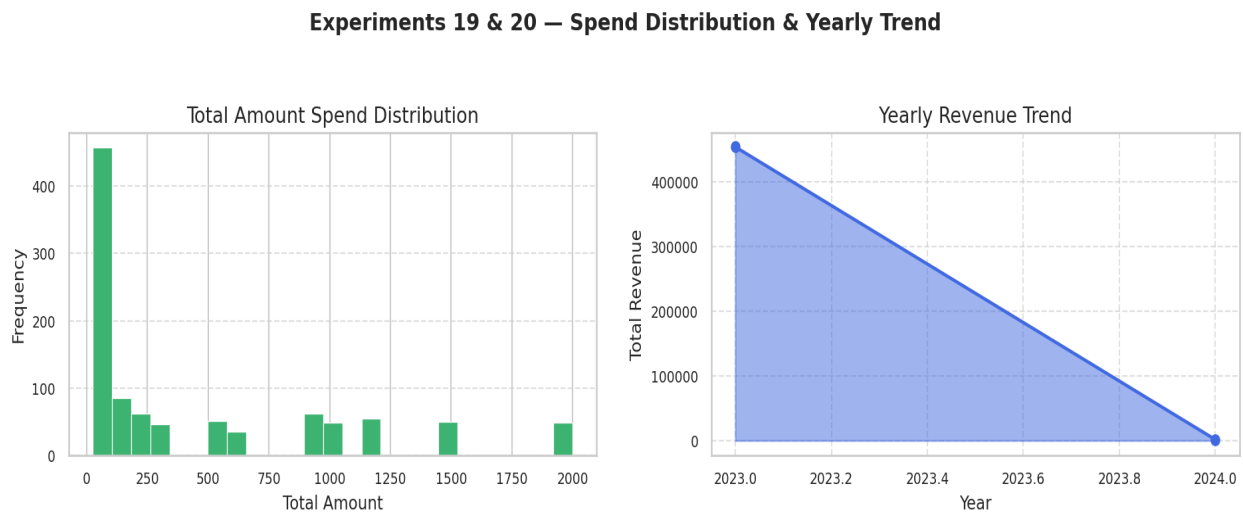

Figure 6.2 — Spend Distribution (histogram) and Yearly Revenue Trend (area chart)

The spend distribution histogram is strongly right-skewed — the majority of transactions are low-value purchases (under ■200), driven by Beauty products. The long right tail represents high-value Electronics purchases. The yearly trend shows that most revenue is from 2023, with a small tail into January 2024.

6.4 Exploded Pie Chart

```
# Pie chart with the largest segment exploded outward
cat_counts = df["Product Category"].value_counts()
explode_vals = [0.15 if c == cat_counts.idxmax() else 0
for c in cat_counts.index]

cat_counts.plot.pie(autopct="%1.1f%%",
explode=explode_vals,
colors=plt.cm.Set2.colors[:len(cat_counts)])
plt.title("Distribution of Product Categories")
plt.ylabel("") # Hide the default y-axis label
```

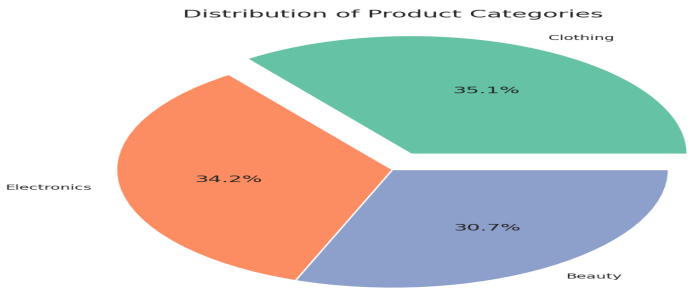

Figure 6.3 — Exploded Pie Chart highlighting the highest-volume Product Category

Using explode to highlight the leading category (Clothing, 35.1%) draws immediate visual attention to the most important segment. The explode parameter takes a list of offset values — 0.15 pushes the target slice outward while 0 keeps others in place.

7. Statistical & ML Concepts Applied

This project demonstrates a spectrum of statistical analysis methods, from basic summary statistics to advanced preprocessing pipelines.

7.1 Univariate Analysis

```
# Numeric univariate analysis
print(df["Total Amount"].describe())
# count=1000, mean=456, std=560, min=25, max=2000

print(df["Age"].describe())
# mean=41.4, median=42, std=13.7

# Categorical univariate analysis
print(df["Product Category"].value_counts())
print(df["Gender"].value_counts())

# Skewness of Total Amount
print(df["Total Amount"].skew()) # Positive = right-skewed
```

7.2 Bivariate Analysis

```
# Numerical vs Numerical: correlation
print(df[["Age", "Total Amount", "Quantity"]].corr())

# Categorical vs Numerical: group comparison
print(df.groupby("Product Category")["Total Amount"].describe())
print(df.groupby("Gender")["Total Amount"].describe())

# Categorical vs Categorical: cross-tabulation
ct = pd.crosstab(df["Gender"], df["Product Category"])
print(ct)

# Visualize bivariate relationship
sns.boxplot(x="Product Category", y="Total Amount", data=df)
```

7.3 Data Distribution Analysis

Metric	What It Measures	Retail Dataset Application
Mean	Average value of the distribution	Mean Total Amount = ■456
Median	50th percentile — middle value	Median Total Amount = ■135 (right-skewed)
Std Dev	Average spread around the mean	Std Dev of Total Amount = ■560 (high spread)
Skewness	Asymmetry of distribution tail	Total Amount is right-skewed (rare big purchases)
IQR	Q3 - Q1: middle 50% of data	Used in boxplots to identify outlier transactions

Kurtosis	Sharpness of the distribution peak	High kurtosis = extreme high-value outliers present

7.4 Categorical Encoding Techniques

```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder

# Label Encoding (binary or ordinal categories)
le = LabelEncoder()
df["Gender_encoded"] = le.fit_transform(df["Gender"])
# Female=0, Male=1

# One-Hot Encoding (multi-class nominal categories)
category_dummies = pd.get_dummies(df["Product Category"], prefix="Cat")
# Columns: Cat_Beauty, Cat_Clothing, Cat_Electronics

# Dummy Encoding (drop first to avoid multicollinearity)
df_dummy = pd.get_dummies(df[["Gender", "Product Category"]],
columns=["Gender", "Product Category"],
drop_first=True)
```

7.5 Visualization Library Comparison

Library	Level	Best For	Used In Project
Matplotlib	Low-level	Full control, custom layouts, subplots	Line, bar, hist, scatter, area, bubble
Seaborn	High-level	Statistical plots, themes, auto-coloring	Boxplot, heatmap, scatter, histogram
Pandas	Mid-level	Quick plots directly from DataFrames	df.plot(), value_counts().plot()

8. Key Findings & Insights

8.1 Product Category Distribution

- Clothing leads with 351 transactions (35.1%), Electronics follows at 342 (34.2%), Beauty at 307 (30.7%)
- The near-equal split across categories indicates a well-balanced retail product portfolio
- No single category dominates — unlike Netflix where Movies vastly outnumber TV Shows (69.6% vs 30.4%)
- Electronics generates the highest per-transaction revenue due to higher unit prices (up to ■500)

8.2 Gender & Demographic Analysis

- Gender split is near-perfectly equal: 51.0% Female (510) and 49.0% Male (490)
- No significant gender bias exists across product categories — all three are purchased roughly equally by both genders
- Customer age is broadly distributed from 18–64, with a mean of 41.4 and median of 42
- No strong correlation between age and spending amount — purchasing power is price-driven, not age-driven

8.3 Spending Patterns

- Mean transaction value is ■456, but the median is only ■135 — confirming heavy right-skew
- The majority of transactions (Q1–Q3) fall between ■60–■900, driven by quantity × price combinations
- Electronics drives the highest-value transactions: a single unit at ■500 × 4 quantity = ■2,000 maximum
- Beauty products (priced at ■25–■50) generate the most frequent but lowest-value transactions
- Price per Unit and Total Amount show near-perfect correlation (≈0.99) — price is the dominant spend driver

8.4 Temporal Trends

- Revenue is distributed evenly across 2023 — no strong seasonal peaks or holiday spikes
- Monthly revenue fluctuates between approximately ■30,000–■50,000 with no clear upward or downward trend
- The dataset covers January 2023 – January 2024, providing a full calendar year of trading data
- Unlike Netflix (peaked in 2018–2019), this retail dataset shows stable, non-exponential growth

8.5 Data Quality

- The retail dataset is exceptionally clean — zero missing values across all 1,000 rows and 9 columns
- No duplicate transactions detected; all Transaction IDs are unique (1–1000)
- Quantity values are bounded between 1–4 — no outlier quantities detected
- Price per Unit takes only discrete values: ■25, ■30, ■50, ■300, ■500 — suggesting fixed price tiers

9. Conclusion

This Exploratory Data Analysis of the Retail Sales Dataset demonstrated the full EDA pipeline — from raw data loading and preprocessing through feature engineering, statistical analysis, and advanced visualization across four experiments.

The project covered 11 distinct chart types (line, bar, histogram, scatter, area, pie, donut, boxplot, heatmap, bubble, and stacked bar), each implemented in both basic and advanced forms using Matplotlib and Seaborn. Scikit-learn's LabelEncoder and train_test_split were applied to prepare the dataset for future machine learning tasks.

The analysis revealed that this retail store serves a balanced customer base — equal gender split, broad age range, and relatively uniform product category demand. Spending is primarily price-driven rather than quantity-driven, with Electronics dominating high-value transactions. These insights form a strong analytical foundation for building predictive models such as customer segmentation, product recommendation engines, or sales forecasting systems.

9.1 Libraries & Tools Used

Library	Version	Purpose
pandas	2.x	Data loading, cleaning, groupby, pivot, datetime operations
numpy	1.x	Numerical arrays, mathematical operations, random sampling
matplotlib	3.x	Low-level plotting: figures, axes, subplots, all chart types
seaborn	0.13.x	High-level statistical plots with automatic aesthetics
scikit-learn	1.x	LabelEncoder, train_test_split, preprocessing pipelines